

Session 3.3

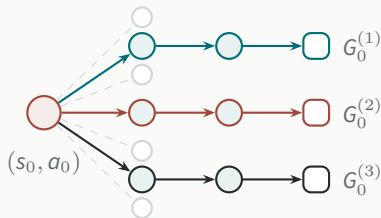
Monte Carlo: Evaluation, Control, and the Data

Average simulated returns to value a rule; then find out what improving a rule from its own data costs



Zhigang Feng

Basic Monte Carlo: Walk N Paths from One Cell, Average the Returns



Basic Monte Carlo for one cell

- 1 **input:** $g, (s, a), N$ paths, horizon T
- 2 **for** $i = 1, \dots, N$:
- 3 simulate T periods: $a_0 = a$, then g
- 4 $G_0^{(i)} \leftarrow \sum_{j=0}^{T-1} \gamma^j r_{j+1}$
- 5 **return** $\hat{Q}^g(s, a) \leftarrow \frac{1}{N} \sum_i G_0^{(i)}$

- ▶ Deck 3.2 ran the one-step end of the dial on a continuous model. This deck starts at the other end — the simplest estimator, on the smallest instance.
- ▶ **Monte Carlo** (MC): average the returns of N simulated paths. Paths are truncated at $T = 300$, and with returns started in the first 150 periods the omitted tail is at most $\gamma^{150} \max |r| \approx 0.003$.
- ▶ **Cost per cell** $O(NT)$, whatever the number of states or shock histories.

Source: Miao & Feng, ch. 11, §11.4.3 — the algorithm “basic Monte Carlo: estimating Q for a single (s, a) ”.

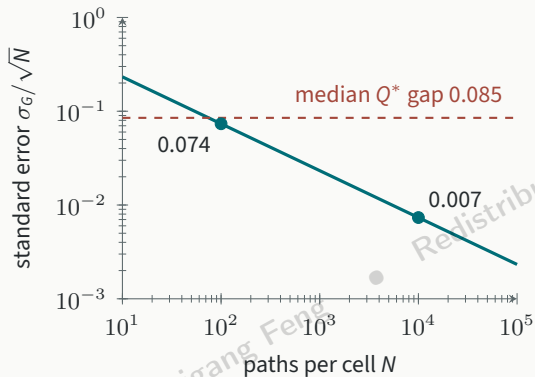
The Exact Q^* Table: the Answer Key for Decks 3.3 and 3.4

$k \setminus k'$	1.50	2.25	3.00	3.75	4.50	5.25	6.00
1.50	0.563	-0.004	-	-	-	-	-
2.25	1.226	1.932	1.441	-	-	-	-
3.00	1.609	2.533	2.946	2.414	-	-	-
3.75	1.879	2.898	3.511	3.735	3.054	-	-
4.50	2.088	3.160	3.864	4.277	4.275	3.620	-
5.25	2.259	3.366	4.122	4.623	4.803	4.784	4.126
6.00	2.403	3.534	4.325	4.877	5.144	5.303	5.257

- ▶ The $z = 0.9$ half of the 7×2 grid. Bold: row maximum, the optimal rule. Best against second best: 0.224 at $k = 3.75$, 0.002 at $k = 4.50$.
- ▶ **A learner must estimate Q more finely than these gaps. This table is the answer key for this deck and for Deck 3.4.**

Source: Miao & Feng, *AI for Economists*, ch. 11, recorded by the book's script; reproduced in the lab, block A. 7×2 grid, $\beta = 0.95$.

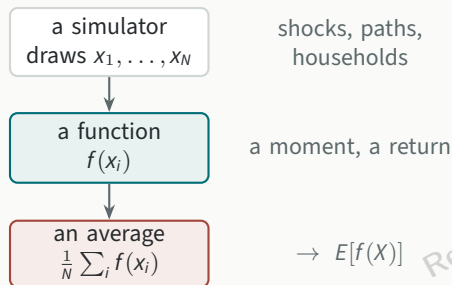
Why a Sample Mean Can Replace the Expectation



- ▶ Law of large numbers: $\hat{Q} \rightarrow Q^g$.
Central limit theorem: the error is about $\sigma_G / \sqrt{N}$.
- ▶ Here $\sigma_G \approx 0.74$: telling best from second best at 95% confidence takes about 400 paths a cell.
- ▶ Ten times the precision costs a hundred times the paths.
- ▶ **The rate depends on neither the dimension of the state nor the number of histories.**

Source: Recorded by the book's script: $\bar{s} = (3.75, 0.9)$, optimal rule, 20,000 paths; 7×2 model, $\beta = 0.95$.

What Monte Carlo Is, in General



- ▶ Monte Carlo estimates *any* expectation by averaging draws, $E[f(X)] \approx \frac{1}{N} \sum_i f(x_i)$. None of the three ingredients is P .
- ▶ **Economists already use this:** the simulated method of moments and indirect inference are the same estimator with a different f .
- ▶ Here f is a discounted return, and the average improves a *rule*, not a parameter.
- ▶ **The estimator is old. What is new is what gets averaged, and what for.**

Source: D. McFadden, *Econometrica* 57(5), 1989; Gouriéroux, Monfort & Renault, *Journal of Applied Econometrics* 8, 1993; Miao & Feng, ch. 11.

Dynamic Programming or Monte Carlo: When to Use Which

	Dynamic programming	Monte Carlo
needs updates error	the law of motion and P every state, every sweep none from sampling	a simulator that produces paths visited cells, after a path ends sampling variance, falling like $1/\sqrt{N}$
cost in dimension d	grows as m^d	does not grow

- ▶ **Stay with dynamic programming** when a state is small enough to sweep, P can be written down, and the error bound is worth having. **Leave it when any one fails.**
- ▶ “Converged” then means two things: an average settling, or a fixed point that stops moving.
- ▶ **Model-free is not error-free: a misspecified simulator passes its error straight on.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “DP and MC compared”.

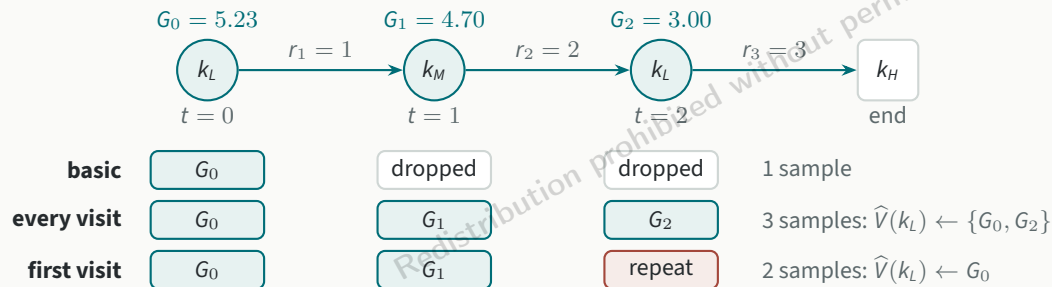
Session 3.3

One Path, Many Samples

Reuse every visit a path makes, and pay for it in coverage



One Path Can Update Every Cell It Visits



- ▶ $\gamma = 0.9$, so $G_0 = 5.23$; by the Markov property G_2 is a sample of $V(k_L)$ too.
- ▶ With $C(s)$ the **visit count**: **every-visit** counts each visit and is biased of order $1/C(s)$ with lower variance; **first-visit** counts only the first and is unbiased.
- ▶ **This session uses first-visit, so the $1/\sqrt{N}$ rate applies as it stands.**

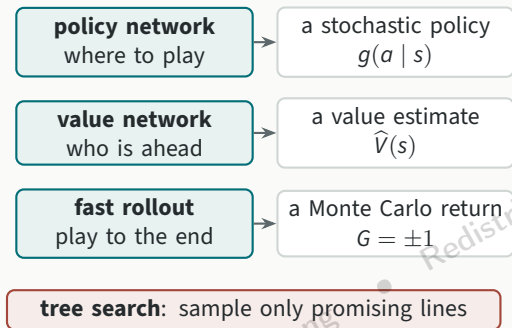
Source: Miao & Feng, *AI for Economists*, ch. 11, "First visit and every visit".

The Price of Reusing Paths: Coverage Becomes the Policy's Job

- ▶ Basic Monte Carlo names every cell as a starting point. Coverage holds by construction, and most of each path is thrown away.
- ▶ Path reuse keeps all of it. But now a cell gets data only if the policy walks there.
- ▶ To evaluate one fixed rule this is harmless: the cells the rule never reaches do not matter for its value where it lives.
- ▶ To *improve* a rule it is decisive: nothing is learned about actions the rule never takes. The second half of this deck is about that.
- ▶ A second limit: G_t contains a long stretch of future shocks, so its variance is high, and nothing can be updated until the path ends.
- ▶ **Low bias, high variance, late updates. Deck 3.4 changes all three at once.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “The trade-off between first visit and every visit”.

AlphaGo: a Policy, a Value, and Monte Carlo Returns



- ▶ Go: about 2.1×10^{170} legal positions, some 250 moves per turn. Ten moves ahead is $250^{10} \approx 10^{24}$ lines.
- ▶ The rules are known. A full backup is still impossible.
- ▶ Policy network: trained on 30 million human positions, then improved by self-play.
- ▶ **Every component is an object from this session. None enumerates the positions.**

Source: D. Silver et al., "Mastering the game of Go with deep neural networks and tree search", *Nature* 529, 2016.

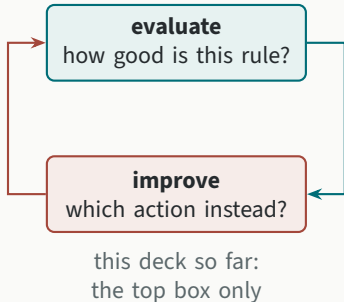
Session 3.3

From Evaluation to Control

Improve the rule from its estimated values, and find out what estimation breaks



Evaluation Was the Easy Half. Now Improve



- ▶ Everything so far values a *fixed* rule. Nothing has tried to change one.
- ▶ Deck 3.2 ran both halves together, one step at a time. Here they are separated, because they fail differently.
- ▶ Wrapping evaluation in an improvement loop is **control**. A warning: in Session 1.4 the control was the choice variable k' — the word's other standard meaning.
- ▶ **The next three slides are what goes wrong when an $\arg \max$ is fed an estimate.**

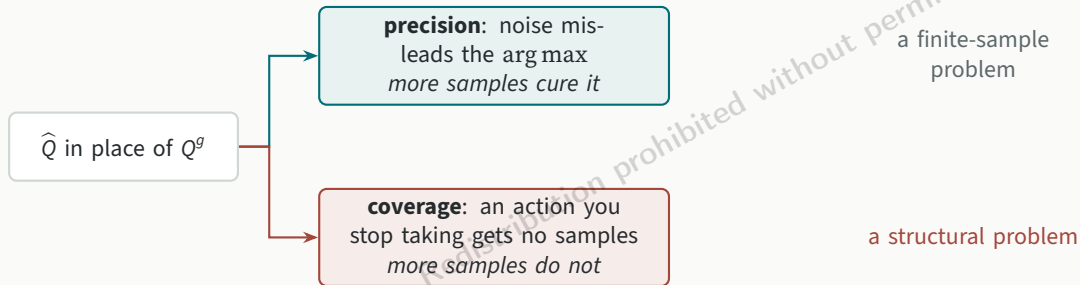
Source: R. Howard, *Dynamic Programming and Markov Processes*, 1960; Miao & Feng, *AI for Economists*, ch. 11.

The Policy Improvement Theorem

- ▶ **Theorem.** In a finite MDP with finite rewards, let g and g' be deterministic. If $Q^g(s, g'(s)) \geq V^g(s)$ at every s , then $V^{g'}(s) \geq V^g(s)$ at every s .
- ▶ The greedy rule $g'(s) \in \arg \max_a Q^g(s, a)$ satisfies the condition. If it brings no gain, g was already optimal.
- ▶ Reading: switching to a better action today only, and carrying on as before, cannot hurt. Do it every day and the whole path is switched.
- ▶ Why: $V^g \leq T^{g'} V^g$; the operator $T^{g'}$ is monotone and a contraction, so iterating it climbs to $V^{g'}$.
- ▶ A finite MDP has finitely many deterministic rules, so exact policy iteration stops at the optimum in finitely many steps.
- ▶ **The theorem assumes the exact Q^g . Monte Carlo hands us $\hat{Q}$.**

Source: R. Howard, *Dynamic Programming and Markov Processes*, 1960; Miao & Feng, *AI for Economists*, ch. 11.

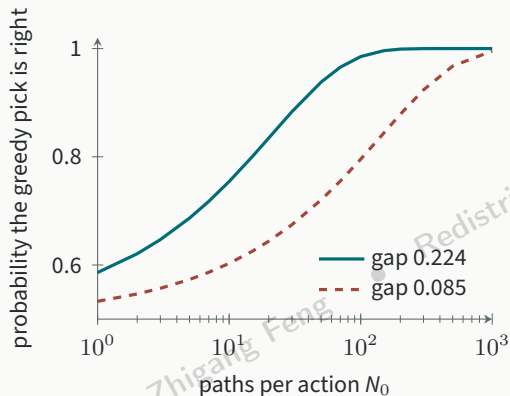
Estimation Breaks the Improvement Theorem — Two Ways



- ▶ The theorem promised: greedy on the *exact* Q^g is never worse. With $\hat{Q}$ it fails in two ways, needing different cures.
- ▶ **One creates mistakes. The other makes them permanent.** Deck 3.2 shows the second on a model with no cells at all: its bad seeds all miss in the same direction, and no amount of data inside the region they visit tells them so.

Source: Miao & Feng, *AI for Economists*, ch. 11, “MC control: where the coverage problem comes from”.

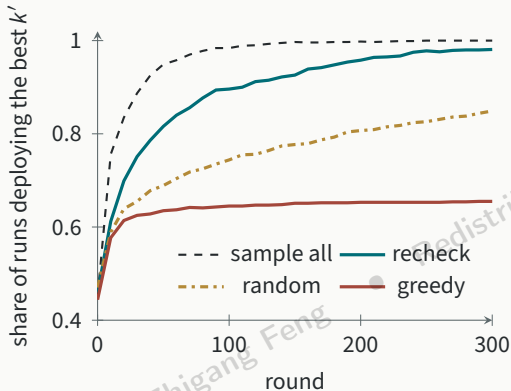
(1) Precision: When Noise Wins the $\arg \max$



- ▶ Two actions, gap Δ , N_0 paths each. The greedy pick is wrong with probability $\Pr\{N(0, 1) > \Delta/(\sigma_G \sqrt{2/N_0})\}$.
- ▶ At $\bar{s}$: right 0.586 of the time at $N_0 = 1$, 0.938 at $N_0 = 50$.
- ▶ Evaluation error $\bar{e}$ can cost $2\bar{e}/(1 - \gamma)^2$ in value: $800 \bar{e}$ here.
- ▶ **Small gaps and few samples: the $\arg \max$ picks noise.**

Source: Normal approximation, $\sigma_G = 0.728$ (book's script). Bertsekas & Tsitsiklis, *Neuro-Dynamic Programming*, 1996.

(2) Coverage: the Actions You Stop Taking



- ▶ One cell, five actions, one path each to start. Each round: deploy the greedy pick, add samples.
- ▶ Sample every action and you reach 0.98 by round 100: there, precision was the whole problem.
- ▶ Pure greedy stalls at 0.66 *for ever*: an unlucky best action is never sampled again, so its error never shrinks.
- ▶ **Rechecking every tenth round costs 0.4 samples a round and reaches 0.98.**

Source: Recorded by the book's script: 2,000 repetitions at $\bar{s}$; "random" tries a random action 10% of the time.

Exploring Starts

Monte Carlo control with exploring starts (first-visit)

- 1 initialise $\hat{Q}$, counts $C \leftarrow 0$, and $g(s) \leftarrow \arg \max_a \hat{Q}(s, a)$
- 2 **for** each episode: draw (s_0, a_0) at random, every feasible pair with positive probability
- 3 simulate T periods: first action a_0 , then follow g
- 4 $G \leftarrow 0$; **for** $t = T - 1, \dots, 0$: $G \leftarrow r_{t+1} + \gamma G$
- 5 **if** (s_t, a_t) is a first visit: $C \leftarrow C + 1$, $\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + (G - \hat{Q}(s_t, a_t)) / C$
- 6 for every s : $g(s) \leftarrow \arg \max_{a \in \mathcal{A}(s)} \hat{Q}(s, a)$

- **Exploring starts:** reset the economy to a random (k_0, z_0) and a random first k_1 . A structural simulator allows that. A real economy does not.
- Only the first step is random. States far from the start stay rare, and with many states random starts waste samples.
- **Clean in theory, and it needs a reset button.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §5.3; Miao & Feng, *AI for Economists*, ch. 11, “Exploring starts”.

ϵ -Greedy: Randomness as a Learning Tool

- ▶ **ϵ -greedy**: with probability $1 - \epsilon$ take the action that looks best; with probability ϵ draw uniformly from the feasible actions. With $\mathcal{A}^*(s) = \arg \max_a \hat{Q}(s, a)$,

$$g_\epsilon(a | s) = \frac{\epsilon}{|\mathcal{A}(s)|} + \frac{1 - \epsilon}{|\mathcal{A}^*(s)|} \mathbf{1}\{a \in \mathcal{A}^*(s)\}.$$

- ▶ Every feasible action keeps probability at least $\epsilon/|\mathcal{A}(s)|$. A state visited again and again has every action tried again and again. No reset is needed.
- ▶ In the growth model the action is next period's capital, so trying other actions moves the economy to other states.
- ▶ The optimal rule is deterministic, and finding it is still the goal. The randomness is there to expose errors in $\hat{Q}$; it can shrink as $\hat{Q}$ improves.
- ▶ **Two prices: you learn the value of a rule that trembles, and only where the economy actually goes.**

Source: Miao & Feng, *AI for Economists*, ch. 11, “ ϵ -greedy”.

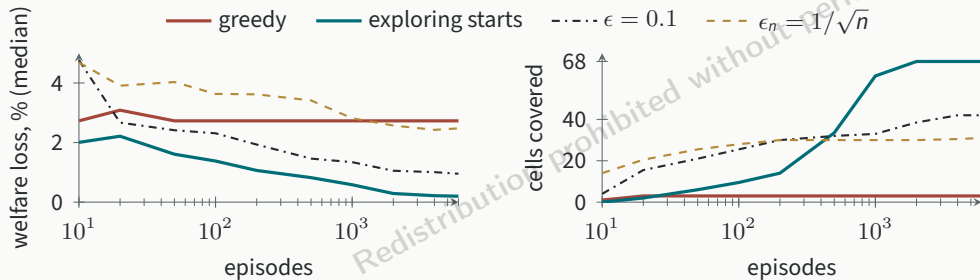
ϵ -Greedy Monte Carlo Control

First-visit Monte Carlo control, on the rule's own data

- 1 initialise $\hat{Q}$, counts $C \leftarrow 0$; choose a start s_0 (it may be fixed) and a schedule ϵ_i
 - 2 **for** episode $i = 1, \dots, N$:
 - 3 from s_0 , draw every action from g_{ϵ_i} built on the current $\hat{Q}$; simulate T periods
 - 4 $G \leftarrow 0$; **for** $t = T - 1, \dots, 0$: $G \leftarrow r_{t+1} + \gamma G$
 - 5 **if** (s_t, a_t) is a first visit: $C \leftarrow C + 1$, $\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + (G - \hat{Q}(s_t, a_t)) / C$
 - 6 **return** $\hat{Q}$ and the greedy rule $g(s) = \arg \max_a \hat{Q}(s, a)$ for deployment
- ▶ Three changes from exploring starts: the start may be fixed; every action is drawn from g_ϵ ; the improved rule is g_ϵ itself.
 - ▶ The rule that collects the data trembles. The rule that is deployed at the end does not.
 - ▶ **Line 5 is a running mean. Deck 3.4 starts from that one line.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §5.4; Miao & Feng, *AI for Economists*, ch. 11, “ ϵ -greedy”.

Measured on the Whole Model: Greedy, Exploring Starts, ϵ -Greedy



- ▶ After 6,000 episodes: greedy 2.73%, 3 cells covered (10 or more samples); exploring starts 0.20%, all 68; $\epsilon = 0.1$ from a fixed start 0.95%, 42.
- ▶ **Loss: the permanent rise in consumption that would compensate for using the learned rule.**

Source: Recorded by the book's script: medians of 20 seeds, one path per cell to start; 7×2 model, $\beta = 0.95$.

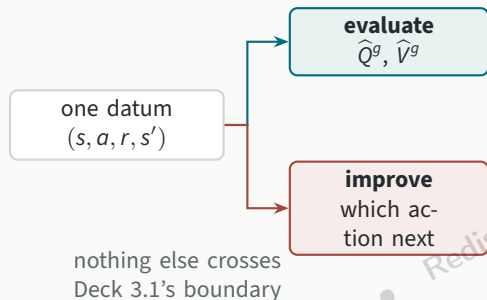
Session 3.3

Who Generates the Data

What a datum is, what it is for, and whose value you end up with



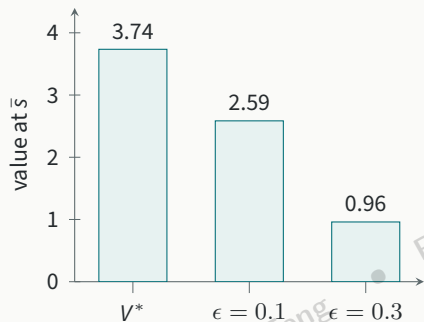
What the Data Are, and What They Are For



- ▶ A datum is one tuple, and it does exactly two jobs. Every method this session is a choice of how to do each.
- ▶ On-policy or off-policy is one question about that pair: **is the rule that generated the tuple the rule being valued?**
- ▶ The economist's version: your sample came from one policy regime and you want the value of another.
- ▶ **Ask of any result: which rule produced these data, and whose value is being reported?**

Source: Miao & Feng, *AI for Economists*, ch. 11, "On-policy and off-policy".

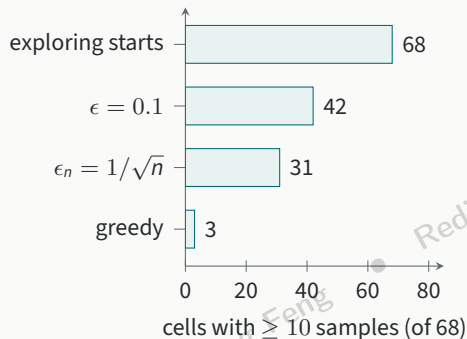
On-Policy and Off-Policy



- **Behaviour policy** b : generates the data. **Target policy** g : the one being learned. Same: **on-policy**. Different: **off-policy**.
- On-policy with $\epsilon > 0$ learns the value of the trembling rule: 5.91% of consumption lost at $\epsilon = 0.1$, 14.88% at 0.3.
- The gap between best and second best shrinks from 0.224 to 0.084.
- **In a real economy trembles are lived. Economists explore inside a simulator.**

Source: Best ϵ -soft rule at $\bar{s} = (3.75, 0.9)$, recorded by the book's script; Miao & Feng, *AI for Economists*, ch. 11.

Data Are Generated by the Policy



- ▶ The rule you follow decides what you see. Its **visitation distribution** $d_g(s, a)$ is the discounted frequency of each pair.
- ▶ Where $d_g(s, a) = 0$, $Q(s, a)$ is not identified: the overlap condition of causal inference.
- ▶ One state alone is a **multi-armed bandit**. Rothschild's firm can stay at a wrong price for ever.
- ▶ **Exploration buys identification.**

Source: Recorded by the book's script (6,000 episodes, medians). M. Rothschild, *JET* 9(2), 1974; Miao & Feng, ch. 11.

Importance Sampling: Reweighting Someone Else's Data

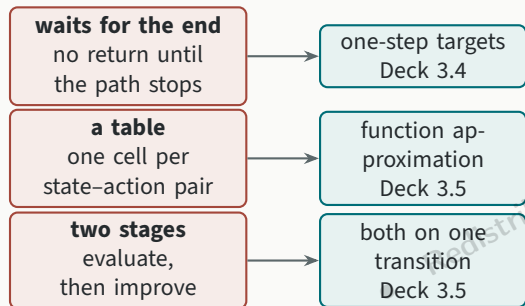
- ▶ Collect with an exploring b , value a greedy g , and correct the mismatch with the **importance ratio** $\rho_{t:T-1} = \prod_{j=t}^{T-1} g(a_j | s_j) / b(a_j | s_j)$.
- ▶ An economist knows it: inverse probability weighting, with the **coverage condition** $g > 0 \Rightarrow b > 0$ as overlap. The weighted form is Hájek's.

horizon T	target	behaviour	ordinary IS	weighted IS	paths with weight > 0
5	0.627	0.621	0.628 (0.026)	0.628 (0.017)	65.9%
20	2.286	1.748	2.312 (0.218)	2.292 (0.072)	19.0%
50	3.424	2.382	3.437 (1.303)	3.422 (0.285)	1.6%

- ▶ One tremble anywhere sets a path's weight to zero. At $\epsilon = 0.3$ and $T = 50$, no usable path is left.
- ▶ **The ratio is a product over the whole path, and the product is the problem. Deck 3.4's Q-learning is off-policy one step at a time, with no product at all.**

Source: Horvitz & Thompson, *JASA* 47, 1952; recorded by the book's script: from $\bar{s}$, 500 paths per estimate, 200 repetitions.

What Tabular Monte Carlo Cannot Do



- ▶ The book closes the chapter with three limits, each naming a piece of what comes next: it must wait for a trajectory to end; a table cannot be stored in high dimension; and evaluation and improvement run as two stages.
- ▶ The third is a way of teaching the method, not a requirement of the loop.
- ▶ **Variance is not on this list;** it belongs to the next chapter.
- ▶ **Next, Deck 3.4: update after one step, and stop waiting for the path to end.**

Source: Miao & Feng, ch. 11, §11.7, the bridge to ch. 12; variance, ch. 12.